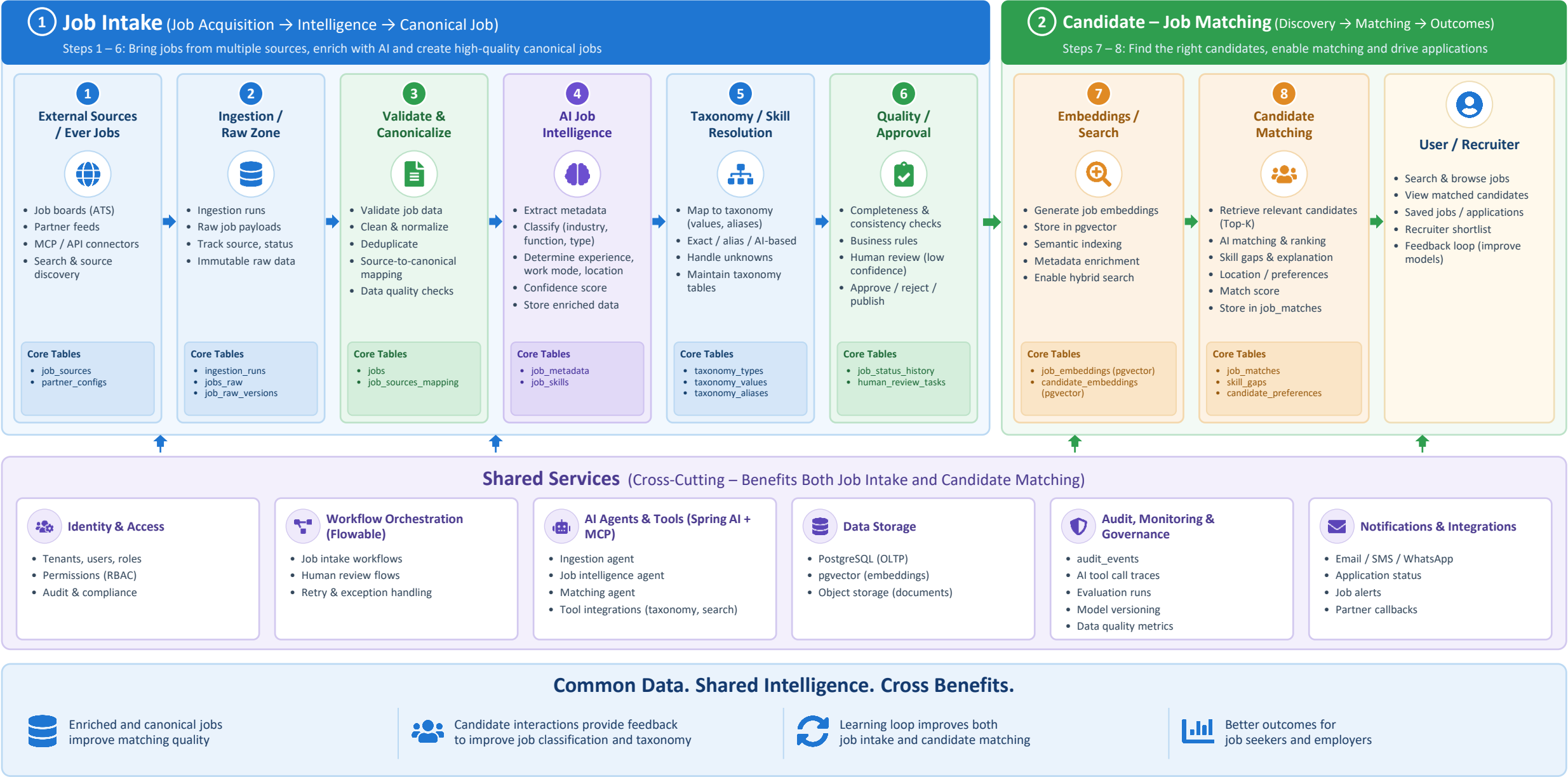




Agent Landscape – What Each Agent Owns

Spring AI + MCP agents, orchestrated by Flowable, across job intake and candidate matching



Ingestion Agent

Steps 1 – 2

- Discovers and pulls jobs from ATS boards, partner feeds and MCP / API connectors on a schedule or on demand
- Lands every payload as immutable raw data with source, version and fetch timestamp preserved
- Tracks run status, retries, rate limits and delta vs. full loads per source

Tools: MCP connectors, scheduler, Apache Camel routes
Writes: ingestion_runs, jobs_raw, job_raw_versions



Canonicalization Agent

Step 3

- Validates mandatory fields and cleans / normalizes titles, salary, dates and locations
- Deduplicates the same role across sources using fuzzy keys plus embedding similarity
- Maps each source field to the canonical job schema while keeping full provenance

Tools: Rules engine, dedup index, normalization library
Writes: jobs, job_sources_mapping



Job Intelligence Agent

Step 4

- Extracts structured metadata and skills from unstructured job description text
- Classifies industry, function, job type, seniority, work mode and location
- Emits a per-attribute confidence score that drives downstream human review

Tools: Spring AI + LLM, prompt templates, extraction schema
Writes: job_metadata, job_skills



Taxonomy Resolution Agent

Step 5

- Resolves extracted values to the controlled taxonomy via exact, then alias, then AI match
- Flags unknown values and proposes new aliases for curator approval
- Keeps taxonomy types, values and aliases versioned and consistent across tenants

Tools: Taxonomy MCP tool, vector lookup, alias cache
Writes: taxonomy_types, taxonomy_values, taxonomy_aliases



Quality & Approval Agent

Step 6

- Scores completeness and consistency, then applies business rules before publish
- Routes low-confidence or rule-breaching jobs to a Flowable human review task
- Publishes, rejects or returns jobs and records the complete status history

Tools: Flowable 7 workflows, rule sets, review UI
Writes: job_status_history, human_review_tasks



Embedding & Search Agent

Step 7

- Generates job and candidate embeddings and stores them in pgvector
- Maintains the semantic index and metadata enrichment used for filtering
- Serves hybrid keyword + vector retrieval to matching and to the search UI

Tools: Embedding model, pgvector, hybrid query layer
Writes: job_embeddings, candidate_embeddings



Matching Agent

Step 8

- Retrieves Top-K candidates per job and Top-K jobs per candidate from hybrid search
- Re-ranks on skills, experience, location and stated preferences
- Produces a match score, skill-gap analysis and a human-readable explanation

Tools: Re-ranking LLM, scoring rules, preference filters
Writes: job_matches, skill_gaps, candidate_preferences



Orchestration & Supervisor

Cross-cutting

- Sequences every agent as a Flowable process with retries, timeouts and compensation
- Enforces tenant isolation, RBAC and per-agent tool call budgets
- Captures traces, evaluation runs and model versions for every agent invocation

Tools: Flowable 7, MCP tool registry, Keycloak
Writes: audit_events, AI tool call traces, evaluation runs

Layered Architecture – Phase 1 (MVP-1) and Phase 2 (Scale-Up)

Same layers throughout – Phase 2 adds scale, isolation and reach rather than replacing the MVP-1 core



Phase 1 → Phase 2 – What Changes and Why

Phase 1 optimises for time to first value; Phase 2 optimises for scale, isolation and channel reach

Capability	Phase 1 – MVP-1	Phase 2 – Scale-Up
Search & retrieval	pgvector semantic search over embeddings, with PostgreSQL predicates for filters and facets on a single index.	OpenSearch / Elasticsearch for faceted keyword search and aggregations, fused with pgvector into hybrid ranking; sharded indices and index aliases for zero-downtime reindex.
Identity & access	Spring Security with JWT; roles and permissions held in the application database.	Keycloak as the IAM system of record: OIDC / SAML SSO, one realm per tenant, RBAC groups, service accounts and token exchange for agent-to-tool calls.
Multi-tenancy	Shared schema with a tenant_id discriminator and application-level filtering.	Enforced isolation via row-level security or schema-per-tenant, per-tenant configuration, rate limits, quotas and encryption keys; tenant-aware caching and indices.
Integration style	Synchronous REST between services; Apache Camel routes and scheduled jobs for source ingestion.	Kafka event backbone with transactional outbox and Debezium CDC; services react to job.ingested, job.published and match.created; DLQ, replay and backpressure handling.
Channels	ReactJS web application for seekers and recruiters, plus transactional email.	React Native mobile app with push, WhatsApp Business API for alerts and apply flows, SMS fallback, and template, consent and preference management.
Workflow & orchestration	Flowable 7 driving job intake and human review tasks in-process.	Clustered Flowable async executors, event-driven signals from Kafka, SLA timers, escalation paths and per-tenant process variants.
AI & agents	Spring AI with MCP tools, per-attribute confidence scoring and human review for low confidence.	Evaluation harness with golden sets, model versioning and A/B routing, feedback-driven re-ranking, prompt guardrails and per-tenant cost budgets.
Runtime & operations	Single AWS environment, containerised services, CI/CD pipeline, basic metrics and audit trail.	Multi-AZ autoscaling, read replicas and Redis caching, blue-green deploys, OpenTelemetry tracing, SLOs and error budgets.